ROUTE Documentation

Version 0.0.34

Rhiddhi Prasad Das

August 27, 2025

Contents

1 Introduction **2**

2 System Overview **3**

2.1 Simulation Manager 3

2.2 TUI Interface 3

2.3 IO Manager 4

3 Installation and Setup **4**

3.1 If you have Source Code 4

3.2 If you have Executable 5

4 Input Output Formats **5**

4.1 Topologies 5

4.2 Configuration 6

4.3 Statistics 6

4.4 Scripting 7

5 Command References **7**

6 Closing Note **10**

1 Introduction

Wireless Sensor Networks (WSNs) form the backbone of many modern distributed systems, powering applications such as environmental monitoring, disaster management, smart cities, and industrial automation. The performance of such networks is deeply tied to the efficiency of their routing protocols, which must balance energy consumption, latency, and reliability while operating under strict resource constraints. Traditional routing approaches often lack the adaptability or scalability required for these dynamic environments, highlighting the need for optimization-driven strategies.

To address this challenge, ROUTE (Routing Optimization Using Tunable Evolution) has been developed as a lightweight, modular, and high-performance simulation framework for exploring and optimizing routing strategies in WSNs. Implemented in C++, ROUTE provides a flexible backend that models key performance metrics—including energy efficiency, communication latency, and reliability—while supporting a range of evolutionary and swarm-based optimization algorithms. This enables researchers to test, compare, and extend novel or existing routing strategies in a reproducible manner.

ROUTE emphasizes both usability and extensibility. It accepts customizable network topologies via YAML-based input files, allows reproducible experiments through TOML configuration, and produces structured JSON outputs for downstream analysis and visualization. Its terminal-based interface (TUI) features command autocompletion for efficient interaction, while its modular architecture makes it straightforward to integrate custom algorithms through a plugin-like mechanism. Planned extensions include a visualization layer and scripting support, broadening its use for both experimental research and academic applications.

By combining computational efficiency with modular design, ROUTE lowers the barrier to prototyping and benchmarking routing protocols for wireless sensor networks. It serves as a practical tool for advancing research on energy-efficient, latency-aware, and reliable routing, offering researchers and students alike a platform to accelerate innovation in the evolving landscape of WSN optimization.

2 System Overview

ROUTE's architecture separates core simulation logic from user interaction, ensuring flexibility, reproducibility, and ease of extension. The framework consists of three main components: **Simulation Manager**, **TUI Interface**, and **IO Manager**. Each component is designed with modularity in mind so that researchers can adapt ROUTE to new optimization algorithms, network models, or experimental workflows without rewriting the entire system.

2.1 Simulation Manager

The Simulation Manager acts as the core engine of ROUTE. It manages the internal state of the wireless sensor network (WSN), executes the optimization algorithms, and evaluates routing strategies against defined fitness metrics such as energy consumption, latency, and reliability. This module is responsible for:

- Initializing the simulation environment from configuration files.
- Managing topology data (nodes, links, energy states).
- Running optimization algorithms (e.g., GA, PSO, TLBO) in a modular plug-and-play fashion.
- Collecting performance metrics and feeding them to the IO Manager.

2.2 TUI Interface

The Text-based User Interface (TUI) provides researchers with an interactive shell to configure, run, and analyze simulations. It is designed for efficiency and minimal overhead while still offering a smooth workflow. Features of the TUI include:

- Command-line autocompletion and inline help for available commands.
- Interact with simulation data.
- Quick configuration of algorithm parameters (population size, iterations, mutation rates, etc.).
- Integration with scripting files (`.rte`) for batch execution of experiments.

2.3 IO Manager

The IO Manager handles all data exchange between ROUTE and the external environment. It ensures experiments are reproducible by enforcing standardized input and output formats. Its responsibilities include:

- Parsing input files in YAML (topologies), TOML (configuration), and RTE (command scripts).
- Exporting results in structured JSON format for post-processing, visualization and statistics.
- Managing logging of simulation events for traceability.

3 Installation and Setup

3.1 If you have Source Code

To build ROUTE from source, ensure that the following dependencies are installed on your system:

- A C++17 compliant compiler (e.g., g++ or `clang++`).
- CMake (version 3.10 or higher).
- yaml-cpp (for YAML parsing).
- toml++ (for TOML configuration parsing).
- nlohmann/json (for JSON output).
- `readline` (for TUI support).
- `matplotlibcpp` (for result visualization).

Once dependencies are installed, compile ROUTE using:

```
mkdir build && cd build
cmake ..
make
```

The resulting binary `ROUTE_linux` will be placed in the `build` directory.

3.2 If you have Executable

If you have a precompiled executable, no installation is required. Simply place the binary in a desired directory and run it directly:

```
./ROUTE_linux
```

Ensure that the input files (YAML topologies, TOML configuration, RTE scripts) are located in the working directory or their paths are specified when running commands inside the TUI.

4 Input Output Formats

ROUTE employs a modular approach to data handling, where each file type serves a distinct role in the simulation workflow. This separation ensures clarity, reproducibility, and the ability to integrate ROUTE with external tools.

4.1 Topologies

`.yaml`
Topologies describe the layout and characteristics of the wireless sensor network. The `.yaml` format is used due to its readability and hierarchical structure, which makes it straightforward to define nodes, their positions, energy capacities, communication ranges, and other attributes. These files can either be created manually or generated by external topology design tools. An example is provided below:

```
nodes:
  - id: 0
    pos: [0, 0]
    energy: 9999
    type: sink
    bandwidth: 5
  - id: 1
    pos: [14, 1]
    energy: 48
    type: sensor
    range: 7
    packets: 8
```

```
    tx_depl: 0.18
    rx_depl: 0.01
    bandwidth: 2
  - id: 2
    pos: [-10, 10]
    energy: 78
    type: sensor
    range: 13
    packets: 8
    tx_depl: 0.18
    rx_depl: 0.01
    bandwidth: 2

environment:
  propagation_speed: 1
```

In this example, the network consists of three nodes: one sink node (acting as the data collection point) and two sensor nodes. Each sensor is defined with parameters such as communication range, packet count, energy level, and depletion rates for transmission (`tx_depl`) and reception (`rx_depl`). The `environment` block specifies global parameters like propagation speed. ROUTE uses these definitions as the fundamental input for simulating and optimizing routing strategies.

4.2 Configuration

`.toml`
Configuration files (planned for later versions) will define high-level simulation parameters, such as maximum runtime, fitness function weights, or random seeds. The TOML format is chosen because it is human-friendly yet more structured than YAML, and widely adopted in configuration management.

4.3 Statistics

`.json`
Statistics files (to be fully defined later) will record simulation results such as energy consumption, packet delivery ratios, latency, and algorithm convergence. JSON is chosen because it is easily machine-readable and directly compatible with data analysis tools and visualization libraries.

4.4 Scripting

`.rte`
ROUTE scripting files automate simulation workflows. The `.rte` format is a lightweight, line-based command language where each line corresponds to a single action in the simulation. This allows batch experiments, reproducibility, and sharing of simulation pipelines. An example script is shown below:

```
# Example ROUTE script (.rte)

# Load a network topology
load topology topology.yaml

# Set optimization algorithm
set algorithm GA

# Configure parameters
set generations 100
set population 50

generate population

# Run optimization
simulate

# Export results
export last results.json
```

This script demonstrates a typical workflow: loading a network topology, configuring the algorithm and its parameters, executing the optimization, and finally exporting results in JSON format.

5 Command References

This section provides a complete overview of all available commands in ROUTE, along with their purpose, arguments, and usage.

load

`load:`
Loads a simulation environment by specifying a topology, configuration, and algorithm. **Arguments:**

- `topology` – Path to the `.yaml` file describing the network topology.
- `config` – Path to the `.toml` file containing simulation parameters.
- `algorithm` – Optimization algorithm to use (e.g., GA, PSO, TLBO).

save

`save:`
Stores the current state of the simulation. **Arguments:**

- `config` – Saves the configuration into a `.toml` file.
- `topology` – Saves the current network topology into a `.yaml` file.
- `convergence` – Stores convergence statistics, indexed by a unique hash.

list

`list:`
Displays information available in the current session. **Options:**

- `algorithms` – Lists all available routing/optimization algorithms.
- `nodes` – Lists all nodes present in the currently loaded topology.

set

`set:`
Configures simulation hyperparameters. **Arguments:**

- `population` – Sets the evolutionary population size.
- `iteration` – Number of iterations to run the optimizer.
- `c_latency` – Weight assigned to latency in the fitness function.

- `c_energy` – Weight assigned to energy in the fitness function.

simulate

`simulate:`
Runs the simulation for a specified number of trials. **Argument:** `<count>` – Number of independent simulation runs to execute.

export

`export:`
Outputs results of the simulation into a JSON statistics file. **Argument:** `<statistics.json>` – Destination file path.

plot

`plot:`
Visualizes results of the simulation. **Options:**

- `topology` – Draws the network topology.
- `network <hash>` – Visualizes a specific optimized routing configuration.
- `convergence <hash>` – Plots convergence of the optimization process.

generate

`generate:`
Generates synthetic networks or configurations for testing. **Arguments:**

- `population` – Creates an initial population of solutions.
- `topology <node_count> <sink_count> <area_size>` – Generates a random topology given number of sensor nodes, sinks, and area size.

compare

`compare:`
Compares performance across multiple configurations. **Arguments:** Two or more `.toml` configuration files for side-by-side evaluation.

help

`help:`
Displays help for a specific command. **Argument:** `<command>` – The command to describe.

route

`route:`
Executes the routing process once, using the current topology, configuration, and algorithm.

exit

`exit:`
Terminates the ROUTE session safely.

clear

`clear:`
Clears the terminal screen to improve readability.

6 Closing Note

ROUTE is an evolving framework, designed to be lightweight, modular, and researcher-friendly. While the current release provides the core set of commands and functionalities described in this documentation, the project will continue to expand with support for new optimization algorithms, additional input formats, and extended visualization capabilities.

Users are encouraged to experiment, adapt, and integrate ROUTE into their own research workflows. Feedback, suggestions, and collaboration proposals

are always welcome. If ROUTE contributes to your academic work, please consider citing this project or acknowledging it in your publications.

Thank you for using ROUTE, and we look forward to seeing the innovative ways it can help optimize wireless sensor networks.